

JetBrains Mono

THE BLUEPRINT & THE LINEAGE

Inter: Mastering JavaScript
Object Constructors and
Prototypal Inheritance

HERO ASSEMBLY SCHEMATIC - REV 1.2

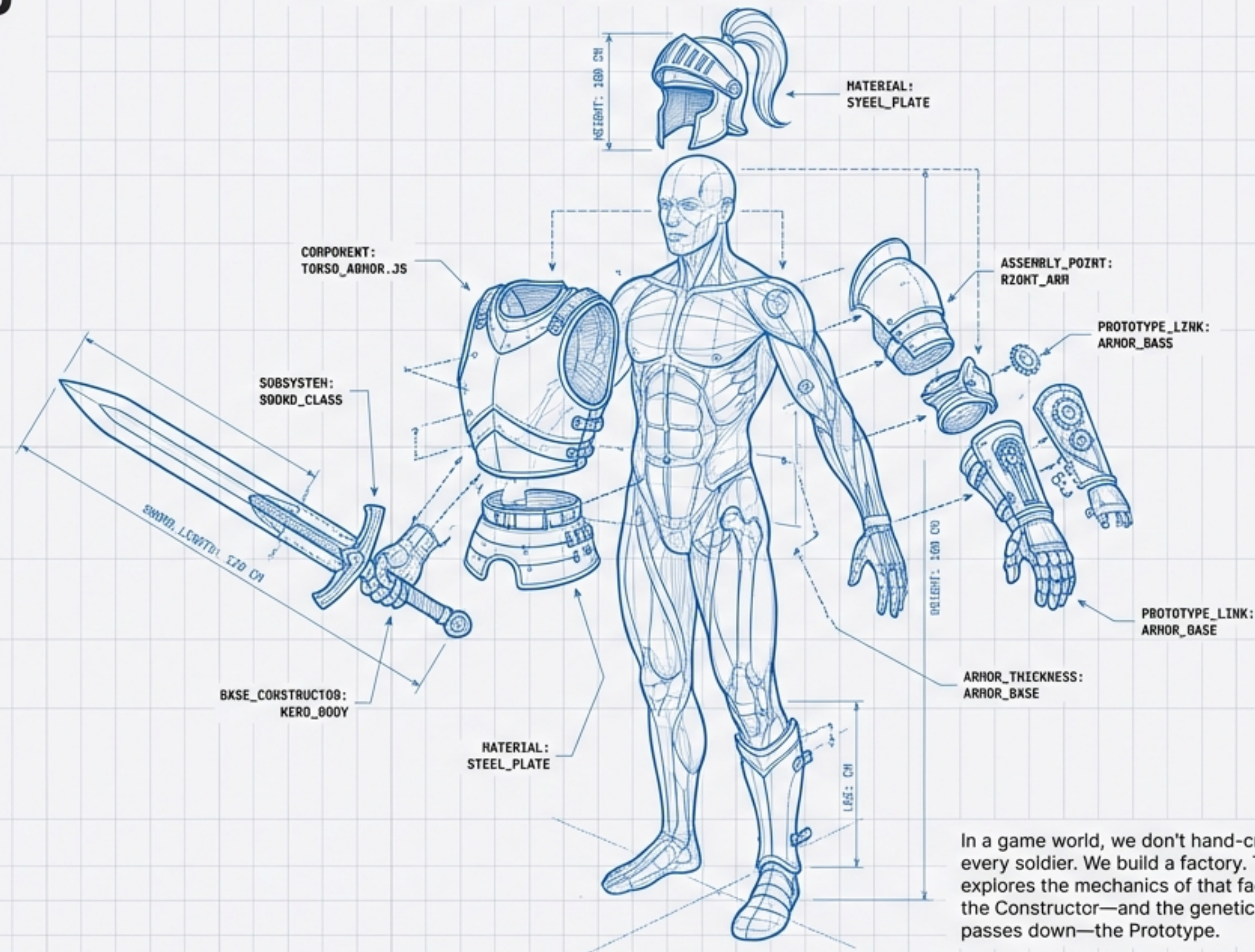
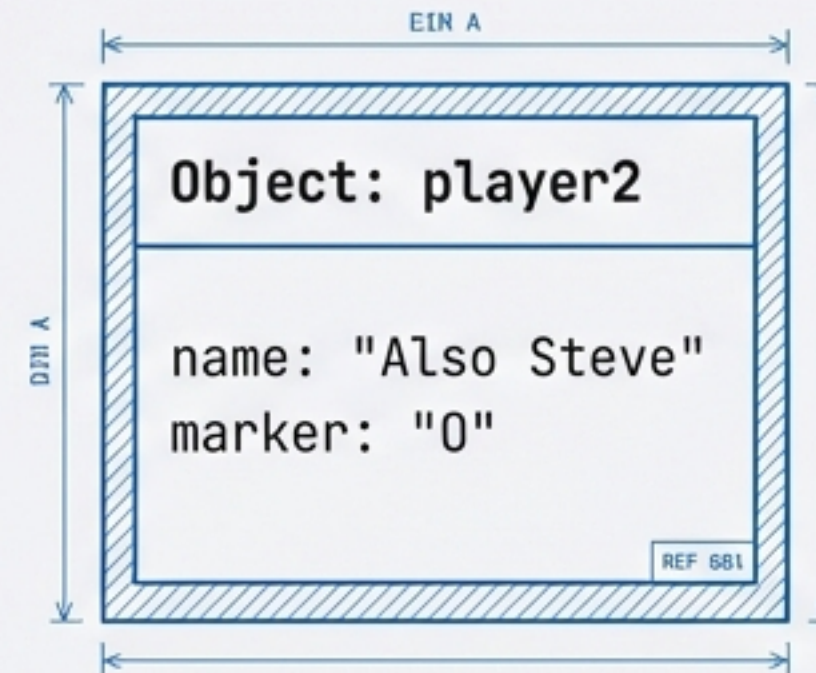


FIG 1.0: THE FACTORY MODEL

THE TRAP OF HARDCODING

```
1 const player1 = {  
2   name: "Steve",  
3   marker: "X"  
4 };  
5  
6 const player2 = {  
7   name: "Also Steve",  
8   marker: "O"  
9 };
```



The Issue: Scaling to 100 players requires duplicating this code 100 times.

The Cost: High maintenance, no shared structure, redundant memory usage.

R1		
R2		
R3		
R4		
REV24W	REV350006	CGT08S

DTAN	SCALE	SCALE
1		

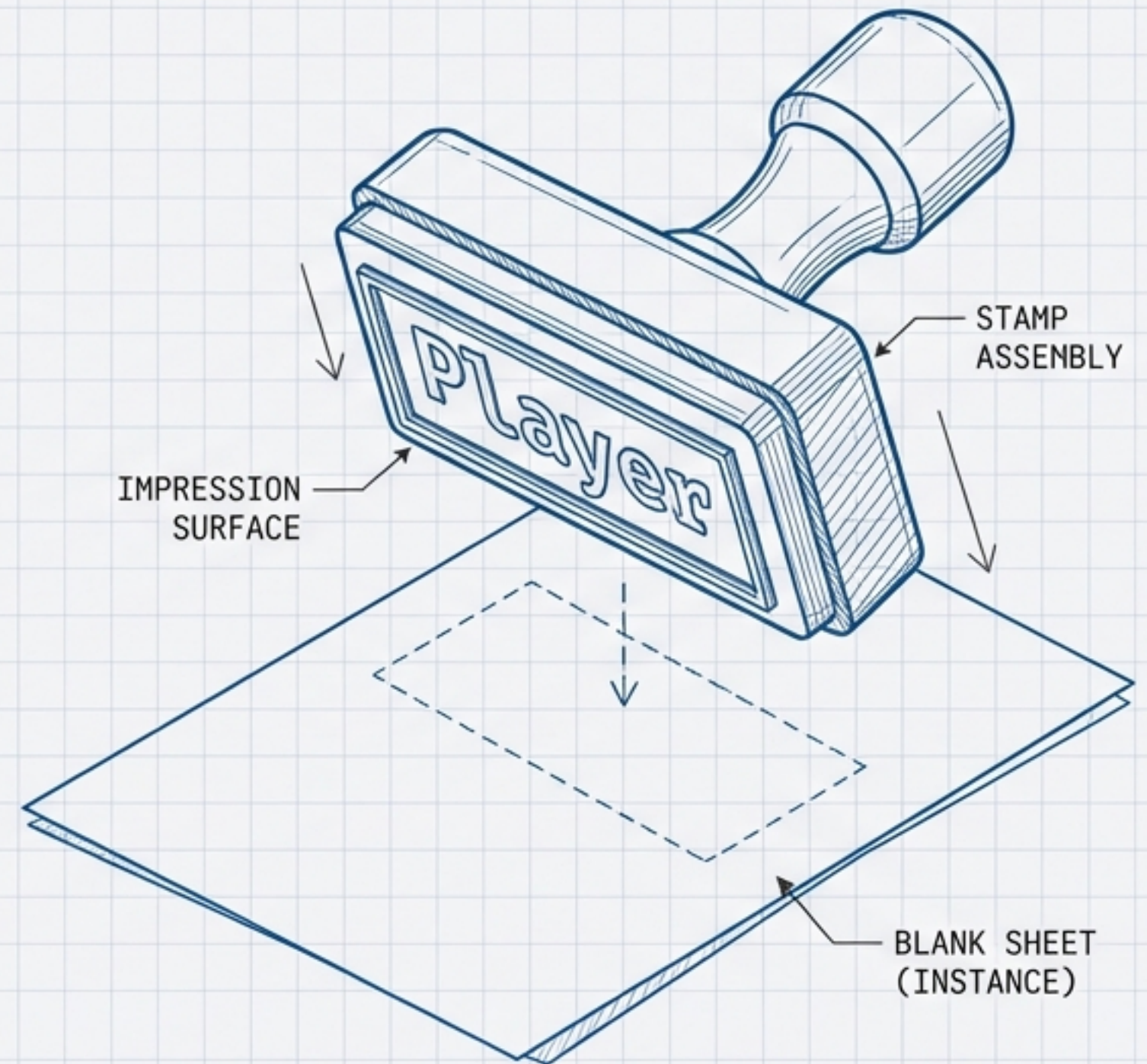
ENTER THE CONSTRUCTOR

Capitalized Convention

```
1 function Player(name, marker) {  
2   this.name = name;  
3   this.marker = marker;  
4   this.sayName = function() {  
5     console.log(this.name);  
6   };  
7 }
```

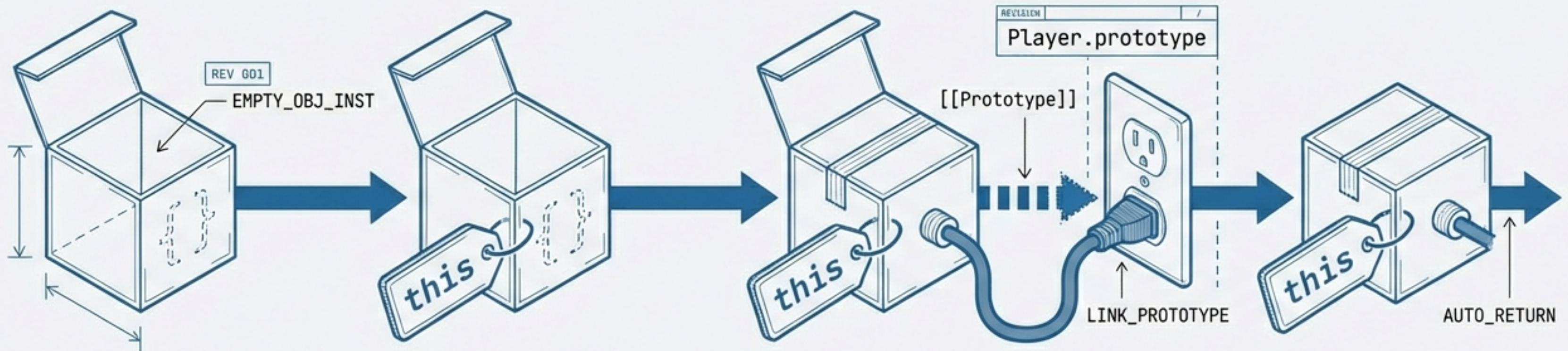
Placeholder for the future instance.

A Constructor is a function that acts as a reusable template. It defines the structure for future objects.



THE MECHANICS OF 'new'

Dispelling the magic: The 4-step internal process.



Creation: Empty object created in memory.

Binding: "this" bound to the new object.

Linking: [[Prototype]] linked to constructor's prototype.

Return: Object returned automatically.

`const player = new Player("Steve", "X");`

Process Execution

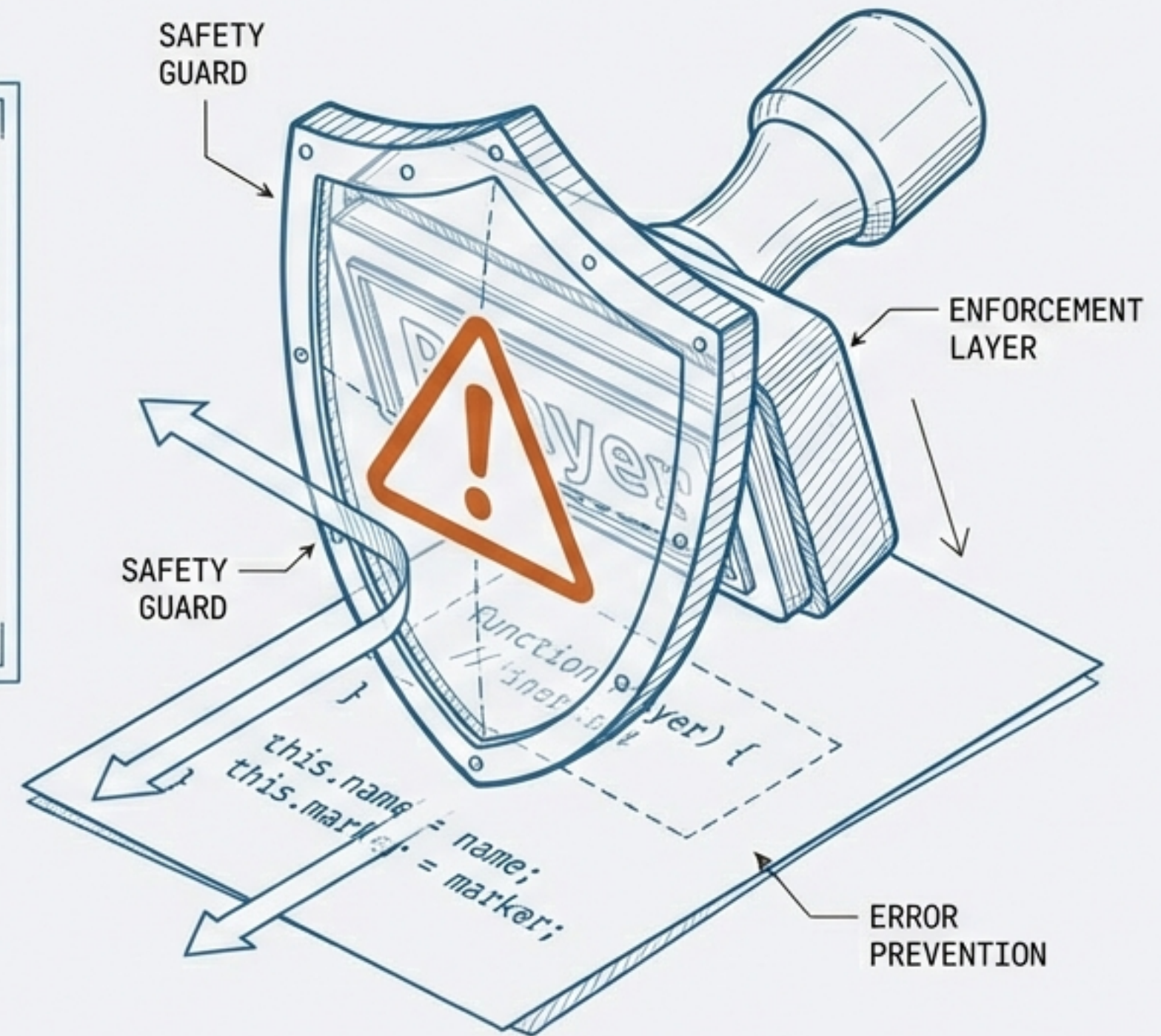
01		
02		
03		
04		
REV 001	REV 000006	C6T006

BEAN	SCALE	SCALE
1		

SAFEGUARDING THE FACTORY

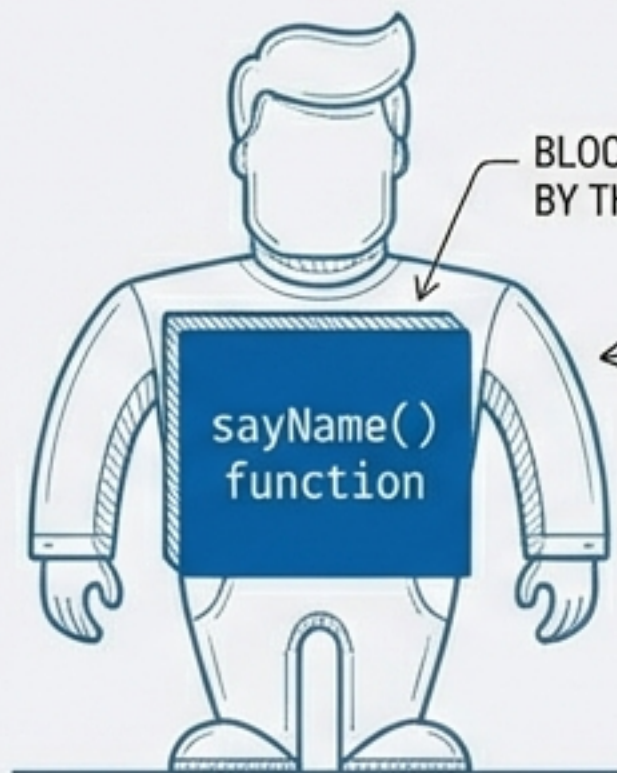
```
1 function Player(name, marker) {  
2   // Safety Check  
3   if (!new.target) { ← 🔍 Enforce 'new' Usage.  
4     throw Error("Must use 'new' operator");  
5   }  
6  
7   this.name = name;  
8   this.marker = marker;  
9 }
```

If `new` is omitted, `this` may bind to the global window object, causing silent failures.
`new.target` allows us to enforce the correct usage of the blueprint.

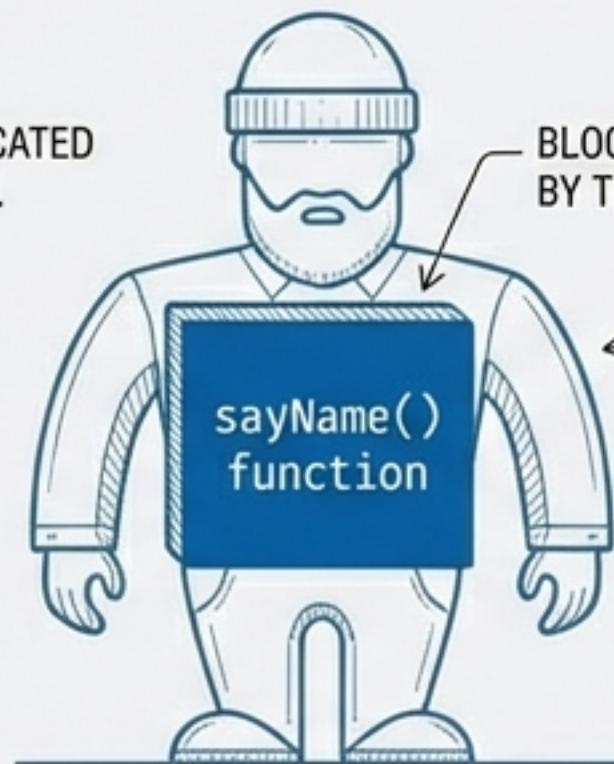


THE MEMORY LEAK IN THE BLUEPRINT

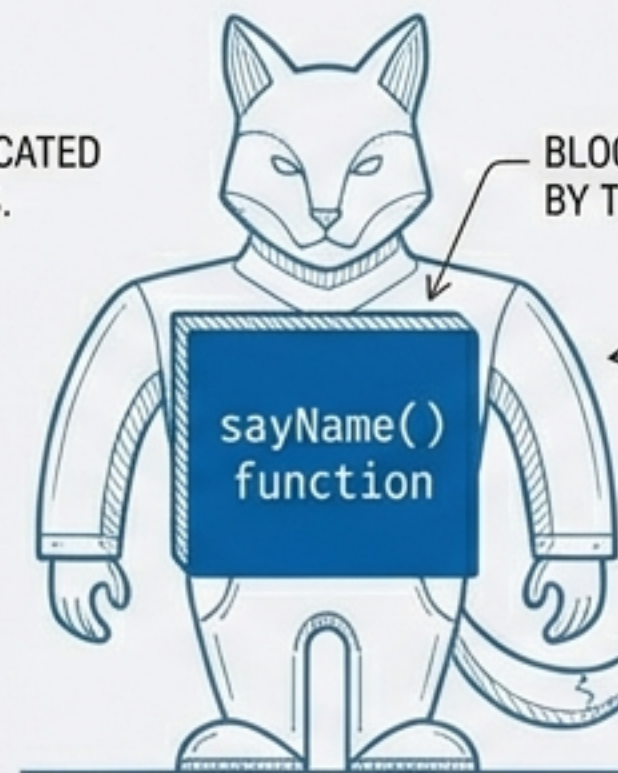
Defining methods inside the constructor replicates the logic for every single instance. This is memory inefficient.



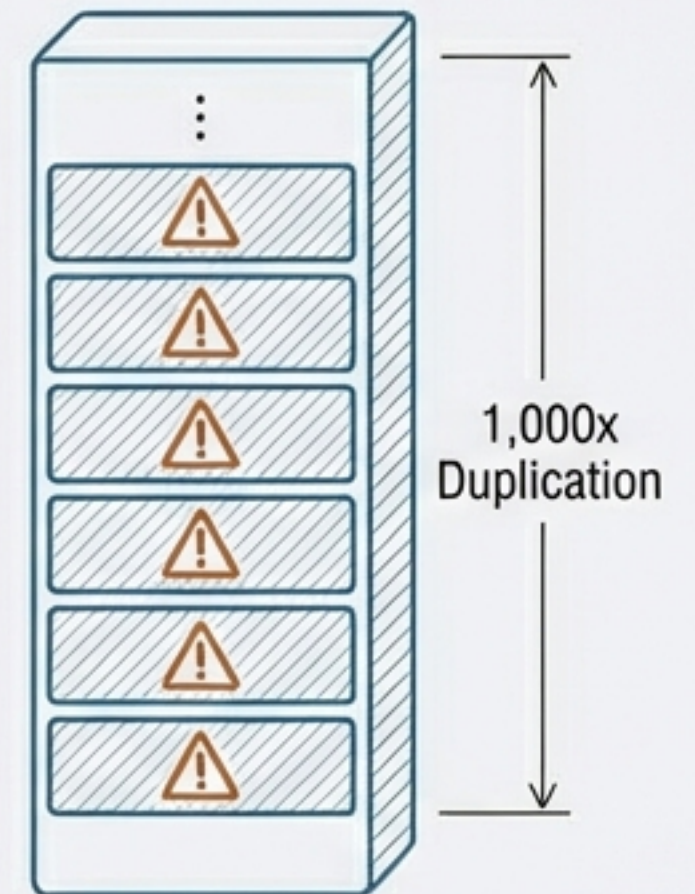
Steve



Carl



Kanin

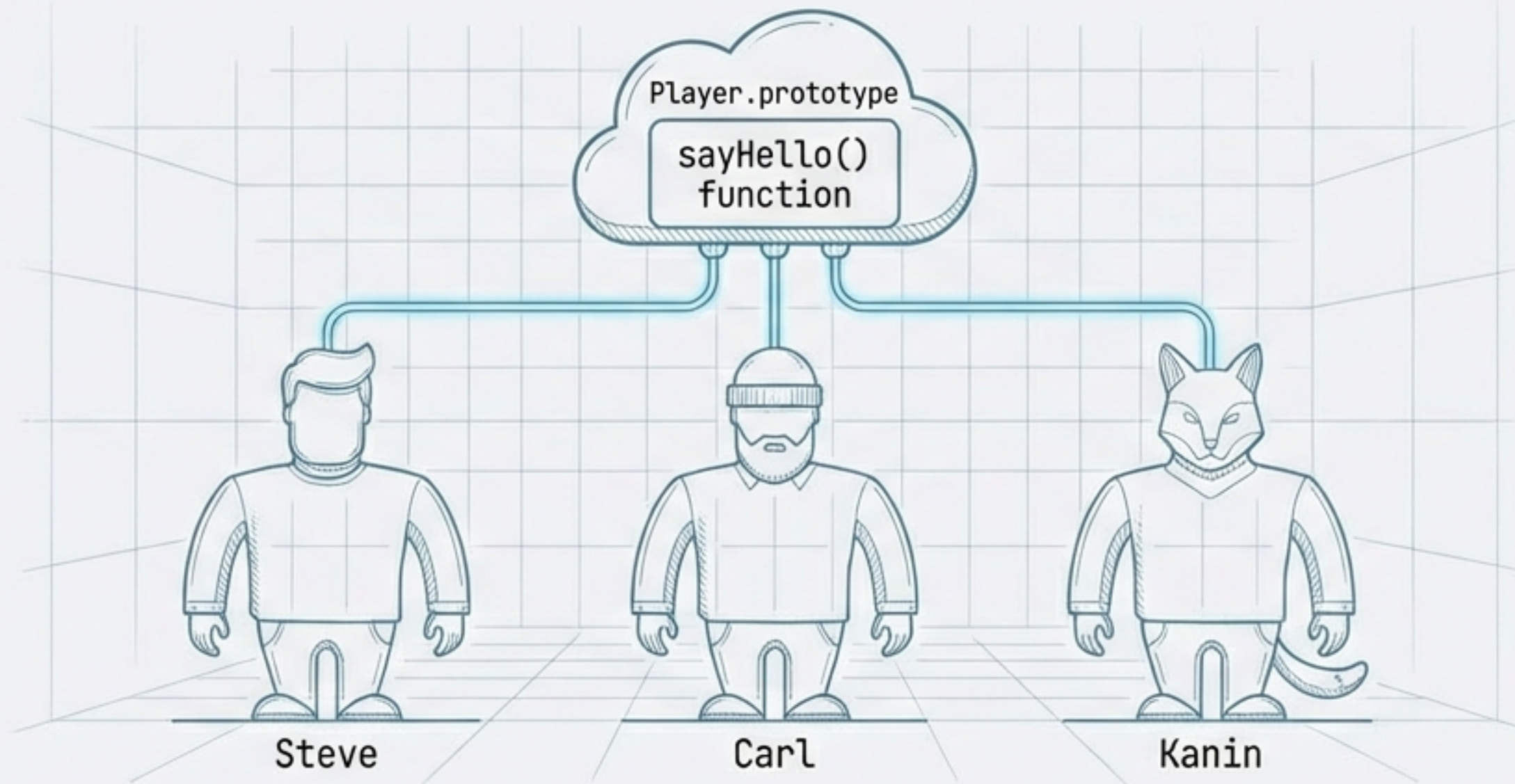


```
function Player(name, marker) {  
  this.name = name;  
  this.marker = marker;  
  this.sayName = function() {  
    console.log("My name is " + this.name);  
  }  
}
```

**1,000 HEROES = 1,000 COPIES
OF THE SAME FUNCTION.**

MEMORY INEFFICIENCY:
REDUNDANT FUNCTION
DEFINITIONS

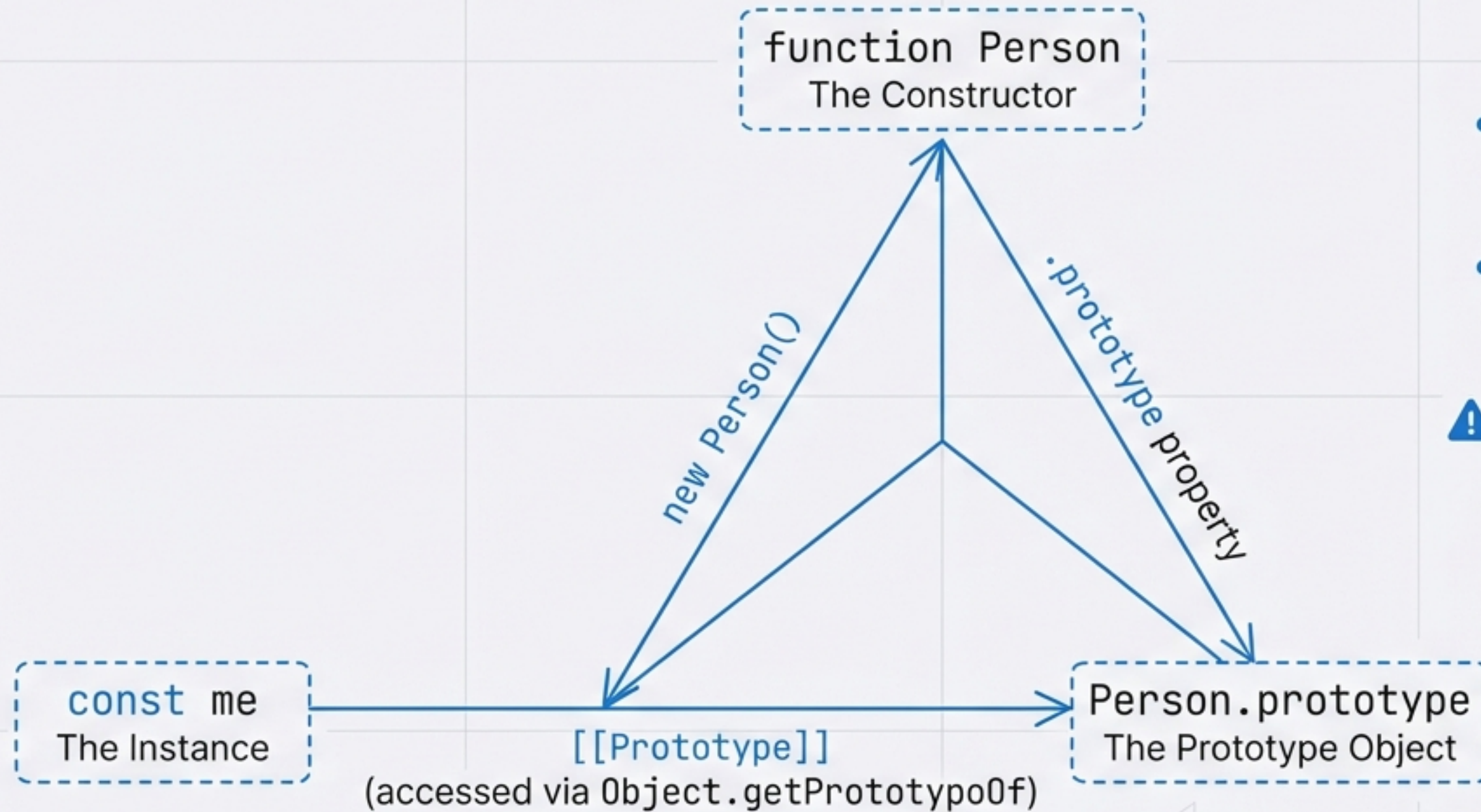
THE PROTOTYPE: SHARED MEMORY



```
Player.prototype.sayHello = function() {  
  console.log("Hello, I'm a player!");  
};
```

1,000 Heroes share exactly ONE copy of the function. Memory usage is optimized.

THE TRIAD OF REFERENCES

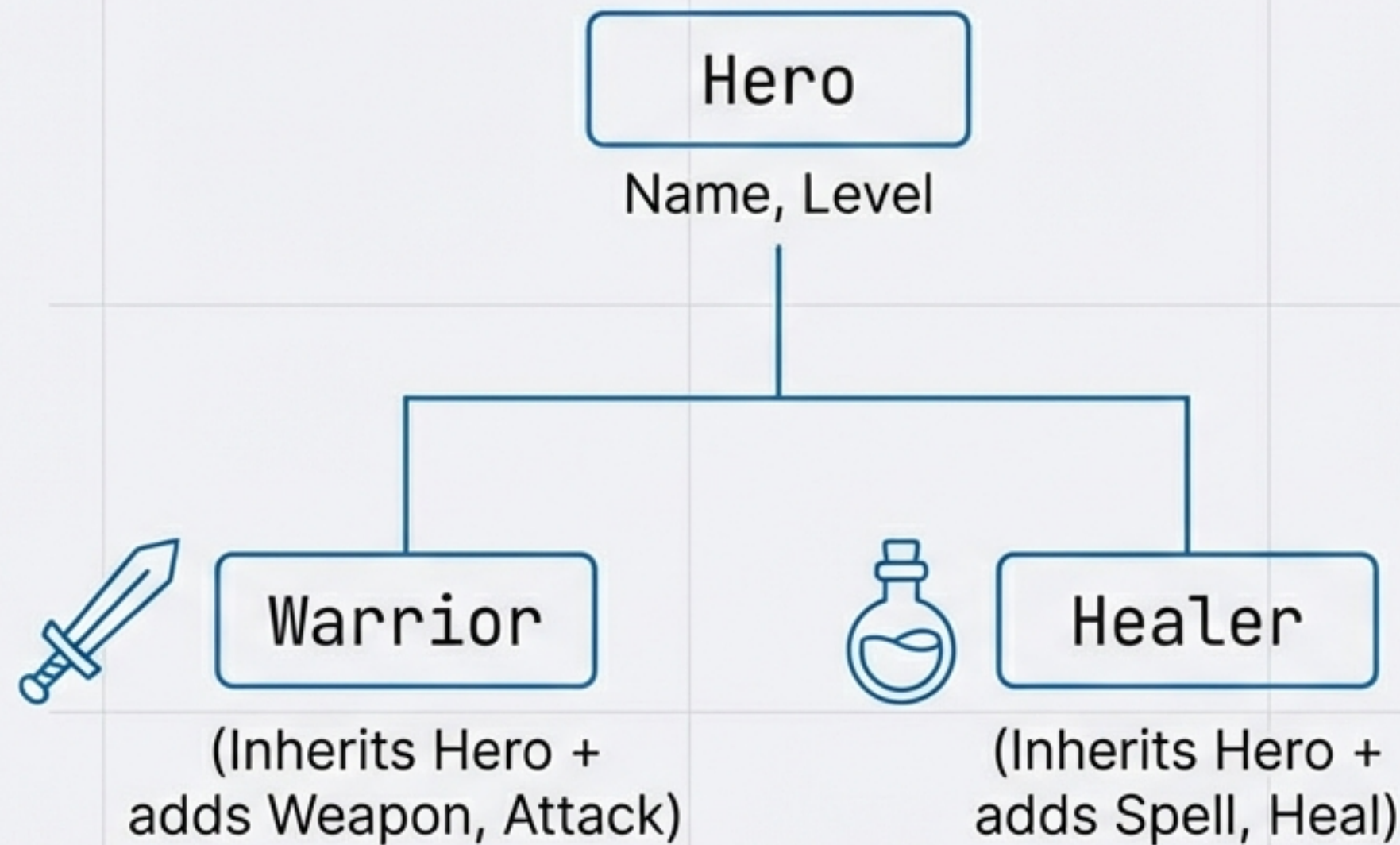


- **Functions** have a `.prototype` property.
- **Objects** have a hidden `[[Prototype]]`.
- ⚠ **Warning:** `__proto__` is legacy/deprecated. Use `Object.getPrototypeOf()`.

```
// Accessing the prototype  
Object.getPrototypeOf(me); // returns Person.prototype
```


EXTENDING THE LINEAGE

Prototypal Inheritance



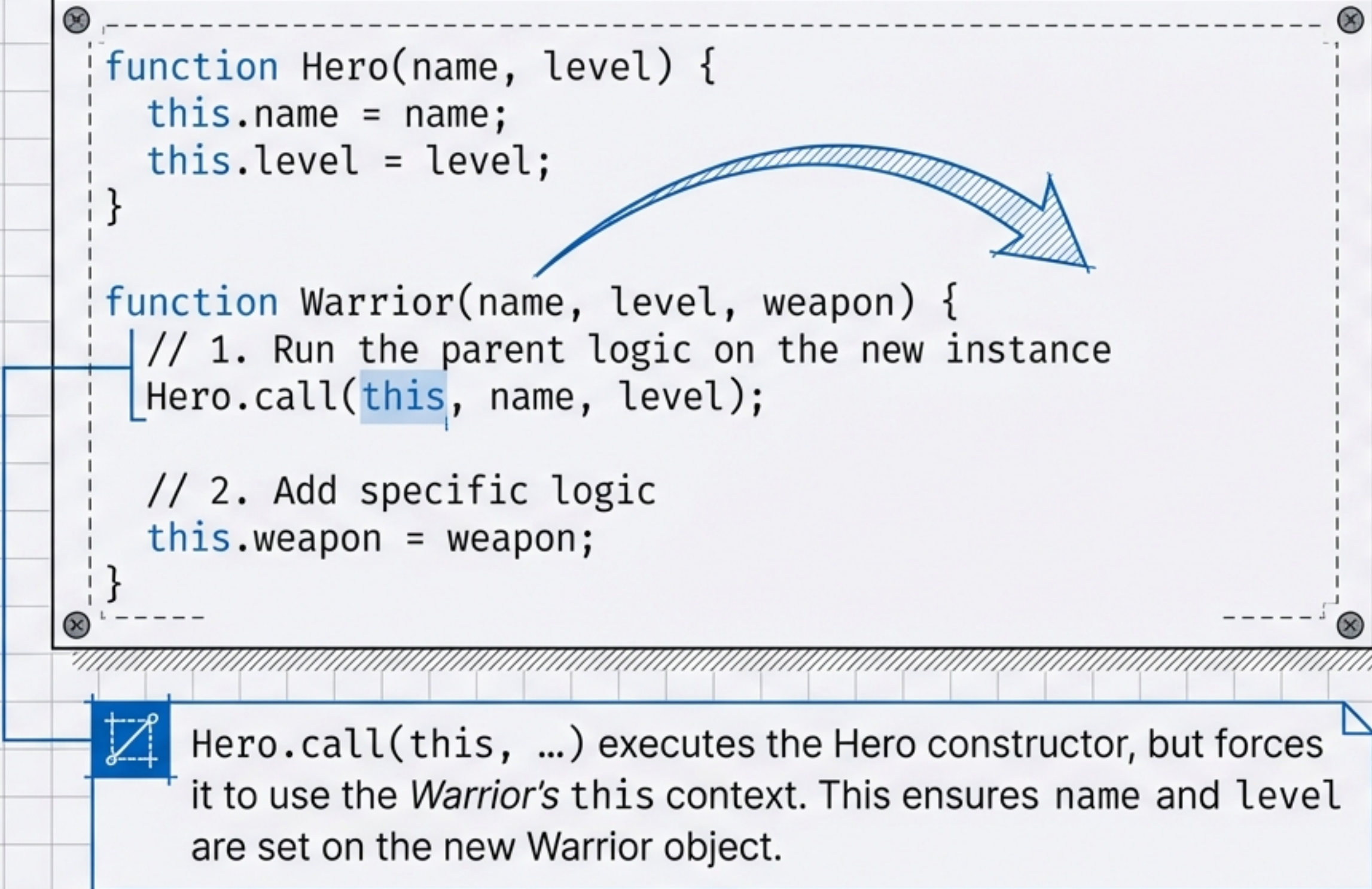
In our RPG, a Warrior *is a* Hero.

We want to reuse the Hero's logic (name, level) but extend it with unique abilities (attack).

We achieve this by chaining prototypes.

STEP 1: CONSTRUCTOR CHAINING

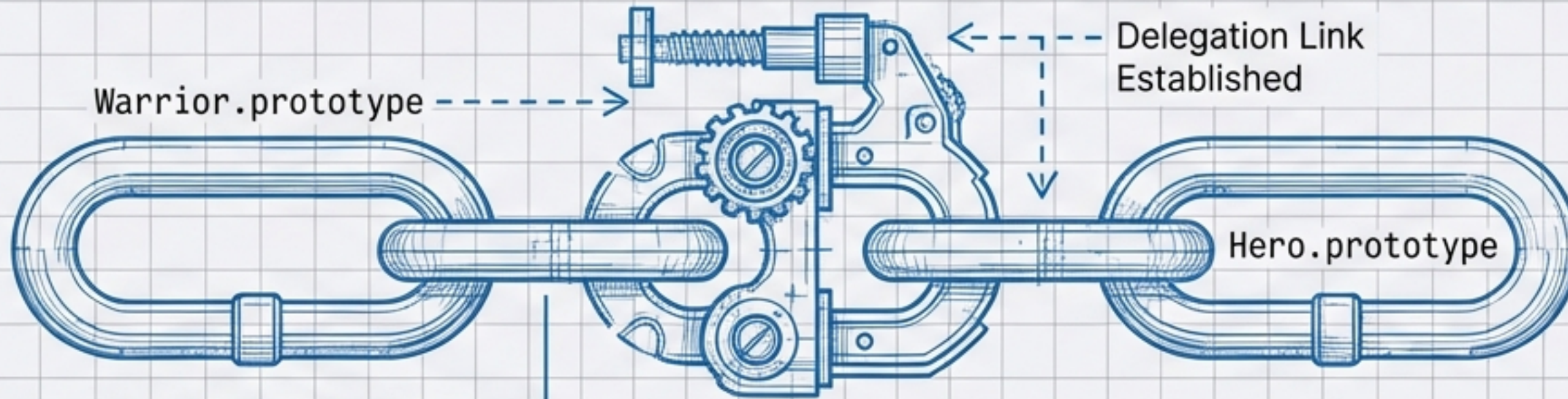
```
function Hero(name, level) {  
  this.name = name;  
  this.level = level;  
}  
  
function Warrior(name, level, weapon) {  
  // 1. Run the parent logic on the new instance  
  Hero.call(this, name, level);  
  
  // 2. Add specific logic  
  this.weapon = weapon;  
}
```



Hero.call(this, ...) executes the Hero constructor, but forces it to use the *Warrior's* this context. This ensures name and level are set on the new Warrior object.

STEP 2: LINKING THE CHAINS

```
Object.setPrototypeOf(Warrior.prototype, Hero.prototype);
```



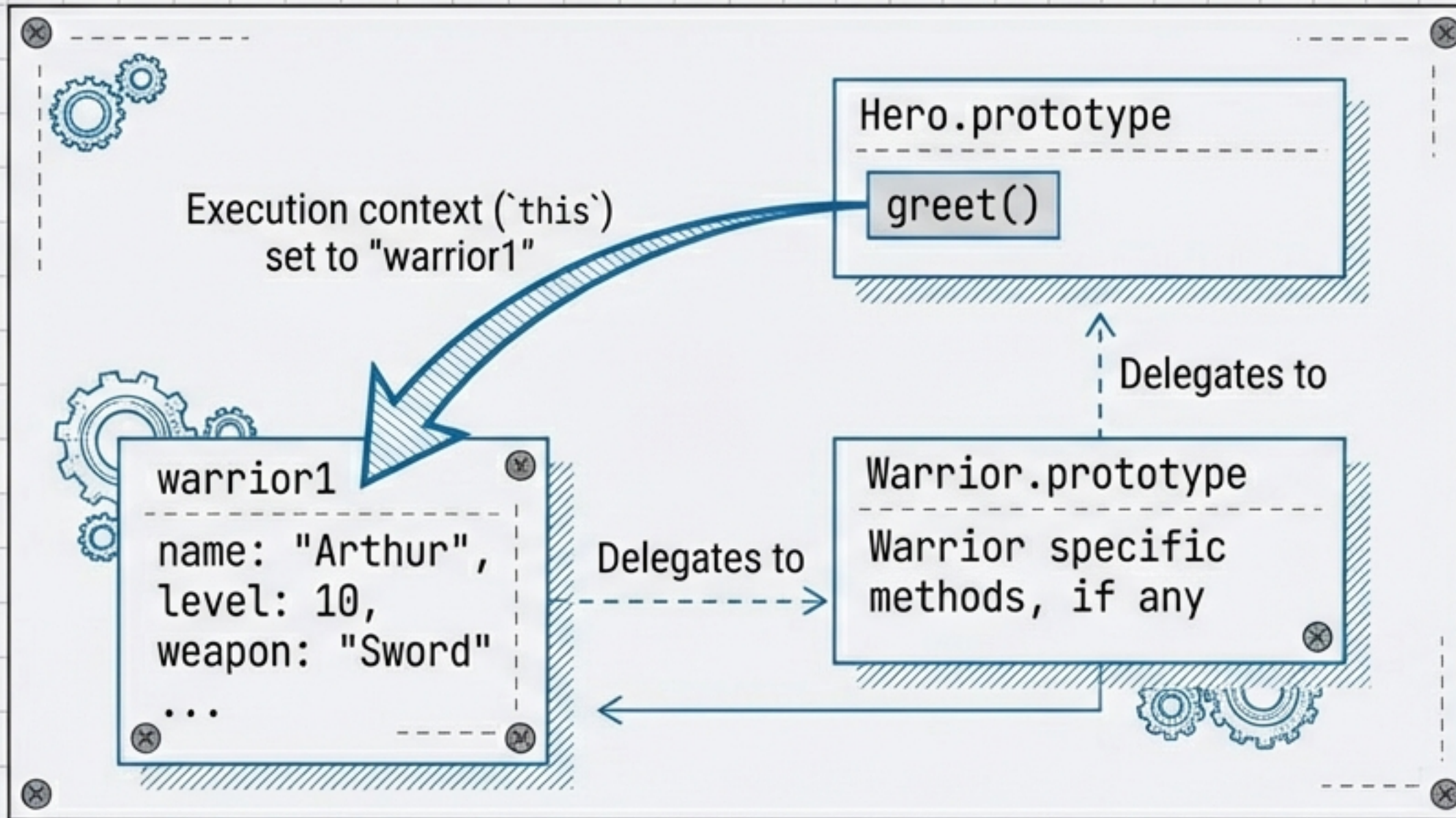
CRITICAL ERROR TO AVOID

```
Warrior.prototype = Hero.prototype; // WRONG ❌
```

Never set them equal. This makes them the *same object*. Modifying Warrior would break Hero. Always use `setPrototypeOf` or `Object.create` to establish a delegation link.

THE IDENTITY CRISIS: WHO IS 'this'?

``this`` is always the object
before the dot at call time.



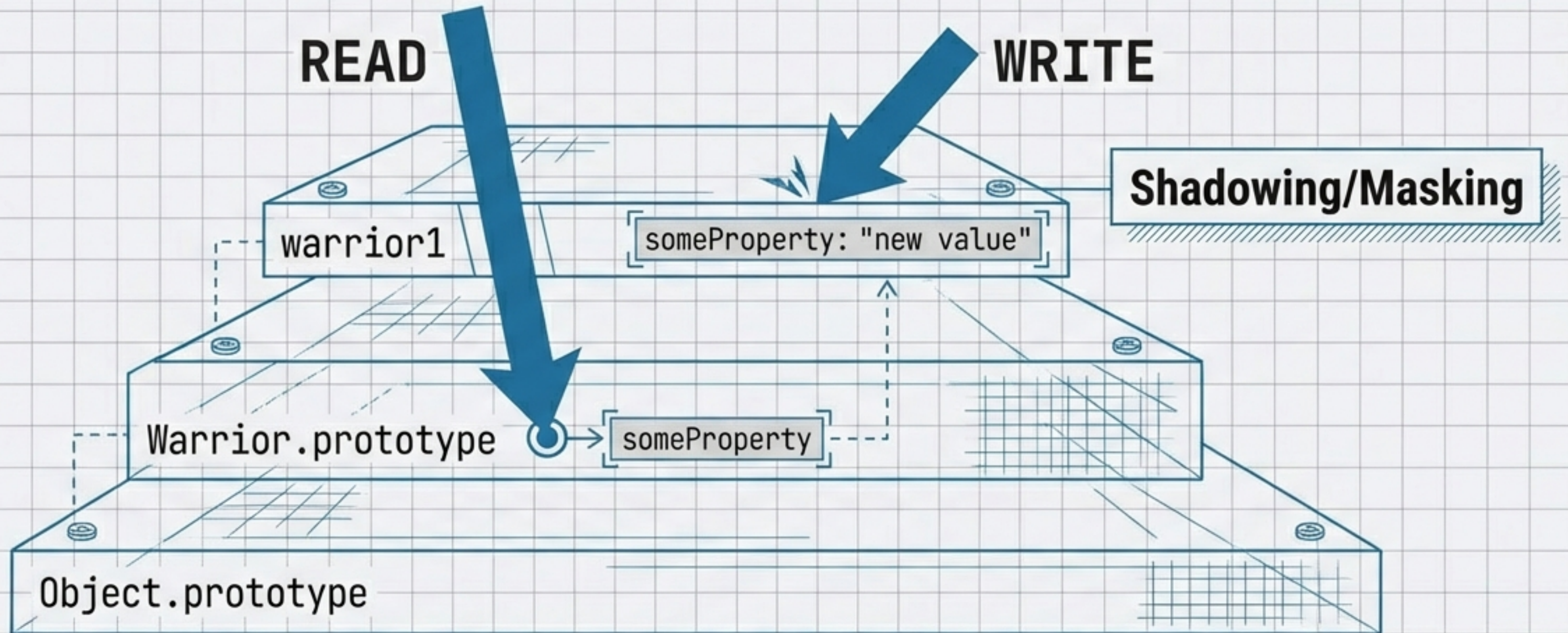
Even though `greet()` is defined on the Hero prototype...

...when we call `warrior1.greet()`, ``this`` refers to `warrior1``.

Result: The method uses the Warrior's name and level.

`warrior1.greet()`

READ VS. WRITE OPERATIONS



Writing to a property ***always*** sets it on the **instance** itself, never the prototype.

WALKING THE CHAIN



for..in loop

Iterates over OWN and INHERITED properties.

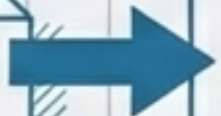
```
if (obj.hasOwnProperty(key)) {  
  ...  
}
```



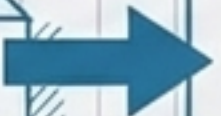
Object.keys()

Returns only OWN properties.

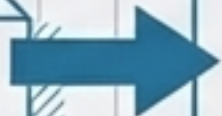
warrior



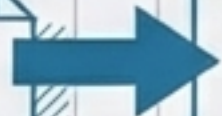
Warrior.proto



Hero.proto



Object.proto



NULL

The chain ends at null. Searching beyond returns `undefined`.

RULES OF THE UNIVERSE

CONSTRUCTION

- ✓ Capitalize constructor names (function Player).
- ✓ Define **properties** in the Constructor (Unique State).
- ✓ Define methods on the Prototype (Shared Memory).
- ✓ Safeguard with `new.target` if needed.

INHERITANCE

- ✓ Call parent constructor: `Parent.call(this, args)`.
- ✓ Link prototypes: `Object.setPrototypeOf(Child.pt, Parent.pt)`.
- ✓ Avoid `__proto__`, use `Object.getPrototypeOf`.
- ✓ Remember: `this` is the object before the dot.